

Búsqueda no informada I

Fundamentos y búsqueda en anchura (BFS)

Joaquín F. Sánchez

Curso de Inteligencia Artificial

2026

Al finalizar la sesión, el estudiante estará en capacidad de:

- representar un problema mediante estados, acciones, estado inicial y objetivo;
- diferenciar la búsqueda informada de la búsqueda no informada;
- explicar la estrategia de expansión por niveles de BFS;
- implementar BFS usando una cola FIFO y control de estados visitados;
- analizar completitud, optimalidad y complejidad del algoritmo.

Situación

Un agente debe encontrar una ruta desde un punto inicial hasta una meta, pero no dispone de una estimación de qué tan cerca está cada estado del objetivo.

¿Cómo puede explorar sistemáticamente sin omitir soluciones?

Pregunta orientadora

Situación

Un agente debe encontrar una ruta desde un punto inicial hasta una meta, pero no dispone de una estimación de qué tan cerca está cada estado del objetivo.

¿Cómo puede explorar sistemáticamente sin omitir soluciones?

Idea central

La búsqueda en anchura explora primero todos los estados cercanos al inicio y avanza nivel por nivel.

La búsqueda como solución de problemas

Un agente basado en objetivos puede seguir este proceso:

- 1 percibir el entorno y determinar el estado actual;
- 2 formular un objetivo;
- 3 formular el problema;
- 4 buscar una secuencia de acciones;
- 5 ejecutar la solución encontrada.

Búsqueda

Proceso computacional que explora alternativas hasta encontrar una secuencia de acciones que conduce del estado inicial a un estado objetivo.

Componentes de un problema de búsqueda

Estado inicial: situación desde la cual comienza el agente.

Estados: representaciones relevantes de las posibles situaciones.

Acciones: operaciones disponibles en cada estado.

Modelo de transición: resultado de aplicar una acción a un estado.

Prueba de objetivo: determina si un estado satisface la meta.

Costo de camino: suma de los costos de las acciones realizadas.

Ejemplo: navegación en una cuadrícula

Estado: posición (*fila*, *columna*). **Acciones:** arriba, abajo, izquierda y derecha. **Objetivo:** alcanzar una celda determinada evitando obstáculos.

Espacio de estados

- contiene estados alcanzables;
- las transiciones representan acciones;
- puede modelarse como un grafo.

Solución

- camino desde el inicio hasta una meta;
- una solución óptima minimiza el costo;
- el camino se reconstruye mediante padres.

Problema $\longrightarrow$ exploración $\longrightarrow$ secuencia de acciones

¿Qué significa búsqueda no informada?

Definición

Una estrategia no informada utiliza únicamente la información incluida en la formulación del problema. No emplea una heurística para estimar la cercanía al objetivo.

- conoce los sucesores de un estado;
- reconoce cuándo alcanza una meta;
- puede usar costos de acciones, según el algoritmo;
- no sabe qué alternativa parece más prometedora.

Estrategias representativas

Búsqueda en anchura, costo uniforme, profundidad, profundidad limitada y profundización iterativa.

Frontera y estados explorados

Frontera

Nodos descubiertos que están pendientes de expansión. La estructura usada para almacenarlos define el orden de búsqueda.

Explorados o visitados

Estados ya descubiertos. Permiten evitar ciclos y trabajo duplicado en la búsqueda sobre grafos.

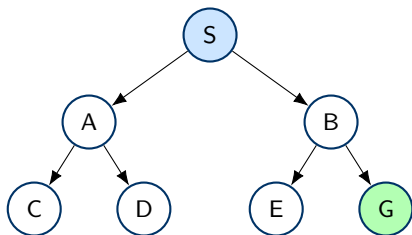
En BFS

La frontera es una cola FIFO: el primer nodo insertado es el primero en salir.

Búsqueda en árbol y búsqueda en grafo

Aspecto	Búsqueda en árbol	Búsqueda en grafo
Memoria de estados	No conserva estados visitados	Mantiene un conjunto de visitados
Ciclos	Puede recorrerlos indefinidamente	Los evita
Estados repetidos	Puede expandirlos varias veces	Evita expansiones duplicadas
Uso recomendado	Espacios sin ciclos o muy controlados	Problemas reales modelados como grafos

Idea de la búsqueda en anchura



- ➊ profundidad 0: S
- ➋ profundidad 1: A, B
- ➌ profundidad 2: C, D, E, G

Orden posible de expansión:

S, A, B, C, D, E, G

La cola FIFO controla la exploración

Paso	Nodo extraído	Cola después de expandir
0	–	[S]
1	S	[A, B]
2	A	[B, C, D]
3	B	[C, D, E, G]
4	C	[D, E, G]
5	D	[E, G]
6	E	[G]
7	G	objetivo encontrado

Invariante útil

Antes de explorar nodos de profundidad $d + 1$, BFS procesa todos los nodos pendientes de profundidad d .

Pseudocódigo de BFS sobre grafos

```
BFS(problema):  
    nodo <- crear_nodo(problema.estado_inicial)  
    si objetivo(nodo.estado): retornar nodo  
  
    frontera <- cola_FIFO([nodo])  
    visitados <- {nodo.estado}  
  
    mientras frontera no este vacia:  
        actual <- frontera.desencolar()  
        para cada hijo en expandir(actual):  
            si hijo.estado no esta en visitados:  
                si objetivo(hijo.estado): retornar hijo  
                visitados.agregar(hijo.estado)  
                frontera.encolar(hijo)  
  
    retornar fallo
```

¿Cuándo se marca un estado como visitado?

Recomendación

Marcarlo cuando se inserta en la cola, no cuando se extrae.

- evita que varios padres inserten el mismo estado en la frontera;
- reduce el consumo de memoria;
- cada estado se procesa, como máximo, una vez.

Detalle de implementación

Para reconstruir la ruta, puede guardarse un diccionario: $\text{padre}[\text{hijo}] = \text{actual}$. Al encontrar la meta, se recorren los padres en sentido inverso.

Implementación base en Python

```
from collections import deque

def bfs(grafo, inicio, objetivo):
    cola = deque([inicio])
    padre = {inicio: None} # tambien funciona como visitados

    while cola:
        actual = cola.popleft()
        if actual == objetivo:
            camino = []
            while actual is not None:
                camino.append(actual)
                actual = padre[actual]
            return camino[::-1]

        for vecino in grafo.get(actual, []):
            if vecino not in padre:
                padre[vecino] = actual
                cola.append(vecino)

    return None
```

Ejemplo de ejecución

```
grafo = {  
    "S": ["A", "B"],  
    "A": ["C", "D"],  
    "B": ["E", "G"],  
    "C": [], "D": [], "E": [], "G": []  
}  
  
print(bfs(grafo, "S", "G"))  
# Resultado: ['S', 'B', 'G']
```

Interpretación

BFS encuentra una ruta con el menor número de aristas cuando todas las acciones tienen el mismo costo.

Propiedades de BFS

Sea b el factor de ramificación y d la profundidad de la solución más cercana.

Criterio	Resultado
Complejidad	Sí, si b es finito
Optimalidad	Sí, si todas las acciones tienen igual costo
Tiempo	$O(b^{d+1})$ en análisis de árbol
Espacio	$O(b^{d+1})$ en análisis de árbol

Principal limitación

La frontera puede crecer exponencialmente. En problemas grandes, la memoria suele ser el recurso crítico.

Otra lectura de la complejidad

En un grafo explícito con lista de adyacencia:

$$T(n) = O(|V| + |E|), \quad M(n) = O(|V|)$$

- $|V|$: número de vértices o estados;
- $|E|$: número de aristas o transiciones;
- cada vértice se visita una vez y cada arista se examina a lo sumo una vez por dirección.

No es una contradicción

$O(b^{d+1})$ describe el crecimiento de un árbol de búsqueda implícito; $O(|V| + |E|)$ describe el recorrido de un grafo explícito.

¿Cuándo conviene usar BFS?

Es una buena opción si:

- se busca la solución más cercana;
- las acciones cuestan lo mismo;
- la solución esperada es poco profunda;
- existe memoria suficiente.

Puede ser inadecuada si:

- el factor de ramificación es alto;
- la meta está muy profunda;
- los costos de acción son diferentes;
- el espacio no cabe en memoria.

- ruta con menos saltos en una red no ponderada;
- menor número de movimientos en rompecabezas sencillos;
- difusión por niveles en redes sociales o de computadores;
- cálculo de distancias mínimas desde un origen;
- exploración de estados alcanzables en autómatas;
- búsqueda de grados de separación entre entidades.

Regla práctica

Si el objetivo es minimizar el número de pasos y cada paso vale lo mismo, BFS es una candidata natural.

Actividad en clase: siga la frontera

Considere el grafo con listas de sucesores en este orden:

$$S : [A, B, C], \quad A : [D, E], \quad B : [F], \quad C : [G], \quad E : [G]$$

Los demás nodos no tienen sucesores. El objetivo es G .

- 1 Escriba la cola después de cada expansión.
- 2 Indique el orden de los estados extraídos.
- 3 Construya el diccionario de padres.
- 4 Determine la ruta que devuelve BFS.
- 5 Explique qué cambia si no se utiliza el conjunto de visitados.

- ❶ ¿Por qué una pila no produce búsqueda en anchura?
- ❷ ¿BFS siempre encuentra la ruta de menor costo? Justifique.
- ❸ ¿Qué función cumple el conjunto de visitados?
- ❹ ¿Por qué BFS puede agotar la memoria antes que el tiempo?
- ❺ ¿Qué diferencia hay entre un estado y un nodo de búsqueda?

- La búsqueda transforma un objetivo en una secuencia de acciones.
- Una estrategia no informada no utiliza estimaciones heurísticas.
- BFS organiza la frontera mediante una cola FIFO.
- Explora el espacio por niveles y es completa bajo condiciones habituales.
- Es óptima para costos unitarios, pero puede consumir mucha memoria.
- En grafos, registrar estados al encolarlos evita ciclos y duplicados.

Próximo tema: profundización de estrategias no informadas